PRODUCT MANUAL + ENGINEERING HANDBOOK

AssignmentTracker

Every feature.
How it works.
Technical reference.

Prepared for Sarveshwar Senthil Kumar

Source-based edition | October 28, 2026

React / Express / MongoDB / PDF tools / Gemini

A COMPLETE PRODUCT AND TECHNICAL REFERENCE

Feature-by-feature instructions, architecture and data flow, guided workflows, engineering notes, design tradeoffs, limitations, source references and implementation inventories.

Implementation descriptions are not runtime-verification claims.

CONTENTS

Find your workflow

Linked entries and PDF bookmarks let you jump to a chapter. The final appendices are extracted from the current source registry and route declarations.

How to use this handbook . . . 4
October 28 edition: recent features and update numbers . . . 5
Product overview . . . 10
Architecture: the whole system . . . 11
Data model and persistence . . . 12
Setup, configuration and deployment boundaries . . . 13
Home, navigation and global search . . . 14
Tasks: everyday planning . . . 15
Task relationships and attached evidence . . . 16
Folders, descriptions and folder media . . . 17
Calendar: a unified view of several sources . . . 18
Calendar: recurrence, colors, categories and undo . . . 19
Public TXST events and the scraper . . . 20
Sports and F1 season import . . . 21
Canvas synchronization and ICS import . . . 22
Document library and supported formats . . . 23
Typed notes and the writing studio . . . 24
Document creation: blank, Markdown and AI . . . 25
PDF editor: annotation fundamentals . . . 26
PDF text: editing, fonts and copying . . . 27
PDF pages, views and productivity tools . . . 28
Pen, touch, mouse and rendering performance . . . 29
Mathematics, calculator, function scanning and graphs . . . 30
OneNote imports and preserving handwriting . . . 31
AI assistant: conversations and user experience . . . 32
AI context, credentials, access and diagnostics . . . 33
AI document editing and PDF cleanup . . . 34
Jobs: links, screenshots, PDFs and local extraction . . . 35
Cover letters: reference voice and template fidelity . . . 36
Resumes and company-specific tailoring . . . 37
Custom workspace and task visualization . . . 38
Study stars and learning workflows . . . 39
Account, authentication and security boundaries . . . 40
Downloads, recovery and the meaning of Save . . . 41

Separate daily-events site and QR poster . . . 42
Guided end-to-end workflow . . . 43
Technical questions: architecture and data . . . 44
Technical questions: PDF, input and performance . . . 45
Technical questions: AI, extraction and trust . . . 46
Engineering case studies . . . 47
Limitations, testing plan and roadmap . . . 48
Troubleshooting and operational recovery . . . 49
Source map and feature inventory . . . 50
Keyboard shortcut reference . . . 51
API implementation inventory . . . 56
Technical quick reference . . . 63

CHAPTER 01

How to use this handbook

This is a source-based guide to Sarveshwar Senthil Kumar's AssignmentTracker, updated on October 28, 2026. It combines a complete product manual, an architecture reference, guided workflows and engineering notes. It describes the implementation without assuming that every feature has passed runtime testing or works with every external service.

Three ways to read it

- Getting started: read Product overview, Architecture and Setup, then follow the guided end-to-end workflow.
- Using a feature: find its chapter in the linked contents or PDF bookmarks for controls, behavior and limitations.
- Understanding the implementation: use the technical questions, engineering case studies, source map and API inventory.

Important distinctions

- Saved in the platform is different from downloaded to a local file. PDF annotations can be stored separately until export.
- Account preferences and records live in MongoDB; some display preferences and recovery drafts live only in the browser.
- Local parsing and calculation are different from AI recognition. The app uses both, with explicit boundaries.
- Undo is scoped to a workflow. There is no universal rollback of every application operation.
- Source inspection establishes implemented structure, not runtime reliability. No application tests, builds, live API probes or browser QA were run to prepare this guide, following the repository's workflow preference.

Reading implementation notes

Each feature chapter separates user-visible behavior from design decisions and external dependencies. Source references identify the relevant modules. Suggested improvements are described as future work, not as deployed capabilities. Performance, extraction quality and integration availability require measurement under representative conditions.

Code takes precedence over older documentation. For example, the assistant can edit supported open documents, notes have a Markdown studio, supported DOCX editing preserves the original package, and Ctrl/Cmd+K opens global search.

Sources: README.md; frontend/src/App.jsx; frontend/src/components/DocumentEditor.jsx; backend/assistantEditor.js.

CHAPTER 02

October 28 edition: recent features and update numbers

This edition retains the complete product manual and technical chapters and adds the recent implementation changes below. Update numbers refer to the numbered project history in LOGBOOK.md, not semantic application versions. This edition covers application changes through Update 282 (commit 5d82697). Update 283 records creation of the separate history logbook. Documentation and source inspection are not evidence that a feature has passed runtime testing.

Quick update map

- Update 271: optional desktop browser inside the study phone.
- Update 272: PDF font matching, compatible Computer Modern fonts and supplied font files.
- Updates 273-275: login handling, implementation corrections and resume template creation.
- Update 276: resume and project interview quizzes.
- Update 277: the Prep workspace.
- Update 278: internet-sourced resume layouts and license credits.
- Update 279: suggested campus events.
- Update 280: attended-events logbook.
- Update 281: future ideas only; not additional implemented functionality.
- Update 282: scrolling and focus when selecting calendar events.

Prep: finding all career preparation tools - Update 277

Open Career in the platform navigation and choose Prep, or use the #/prep route. The workspace contains Interview practice, Resumes & templates, Cover letters, and Saved jobs. These are working tool panels, not just links to unrelated pages. Open a panel to load its tool; previously visited panels remain mounted while you switch within Prep.

Switching Prep tools keeps your current quiz answers and feedback in memory. Leaving Prep clears the quiz session. Saved resumes, templates, letters and jobs remain account records. The standalone Resumes, Cover letters and Saved jobs pages remain available. The product handbook is this document; it is distinct from both the event attendance log and the developer update history.

Sources: frontend/src/components/PrepPage.jsx; frontend/src/components/AppNavigation.jsx; frontend/src/App.jsx.

Create and choose resume templates - Updates 274-275

Open Prep, then Resumes & templates, then Create a resume from a template. The built-in choices are Classic, Modern, Compact and Academic. Their differences include serif versus sans serif typography, contact alignment, margins, spacing, section rules and section order. Preview cards illustrate the layout; they do not promise identical pagination for every amount of text or installed font.

- Select a layout and enter your name and contact details.

- Fill in Profile, Experience, Education, Projects & research and Skills as relevant. Your content remains in the form when you switch layouts.
- Start a line with a hyphen and a space to create a bullet. Empty sections are omitted.
- Select Create & use to generate an editable Word document and upload it to your account. This step does not use AI or require a Gemini key.
- The saved file appears in Resumes and Documents and is selected in the tailoring tool below. Use as template on another saved resume selects that file as the layout source.

The generator uses the existing docx dependency to build a letter-size Word document from your supplied facts. It does not invent experience, import your profile automatically or guarantee a single page. AI tailoring remains a separate review-and-save workflow that preserves the chosen template structure and uses only supplied applicant evidence.

Sources: frontend/src/lib/resumeTemplates.js; frontend/src/components/ResumeTemplateGallery.jsx; frontend/src/components/ResumesPage.jsx; frontend/src/components/CoverLetterGenerator.jsx.

Internet-sourced resume layouts - Update 278

In the same picker, choose From the web. Jake's Resume is a centered, education-first serif adaptation with fine section rules. Bajaj Developer uses a left-aligned header and compact ruled sections. These are editable Word adaptations of MIT-licensed LaTeX layouts, not official Word releases or pixel-identical reproductions.

Choose either layout, enter your own content, and use Create & use as with the built-in templates. Original template and MIT license links are shown for the selected layout. Expand Credits & license notices to read attribution. License text is also retained in the generated Word document's description metadata. The original authors' personal resume data is not inserted into your resume.

Original projects: https://github.com/jakegut/resume and https://github.com/sb2nov/resume. Local attribution: frontend/src/assets/resume-template-licenses.txt. Layout metadata and DOCX output: frontend/src/lib/resumeTemplates.js.

Resume interview quizzes and project deep dives - Update 276

Open Prep, then Interview practice. Alternatively, use Quiz me on a saved resume. Choose Resume interview to discuss the experience on a resume, or Project deep dive to focus on a particular project. Select an available source document or paste at least 40 characters of context. For a project, explain your contribution, decisions, technologies, challenges and results; if selecting a resume, name the project to focus on.

- Choose Introductory, Standard or Challenging difficulty and 3, 5, 8 or 10 questions.
- Start practice to generate questions through your saved Gemini API key.
- Answer one question at a time. Show hint offers a nudge before answering.
- Get feedback reviews the current answer and returns a coaching score from 0 to 5, constructive feedback, strengths and improvements. Example answer structure is expandable after review.
- Previous, Next question and Skip for now allow revisiting questions. Changing an answer clears its old feedback so it is not presented as a review of the new wording.
- View session summary shows reviewed-answer counts and the average available coaching score. Start another session clears the current questions, answers and feedback after the explicit action.

Only the selected document, supplied context, saved text edits and answer/question data needed for the request are sent. Unsaved editor changes are not included. Document ownership and AI category permissions are checked on the backend; generation must be enabled and a Gemini key must be configured in Account. A project file

outside the resume category follows that file category's AI permission. Source material is treated as untrusted data, and the prompt prohibits inventing applicant achievements. These controls do not guarantee perfect grounding or reliable scoring: review all feedback against your actual experience. Scores are practice guidance, not hiring predictions.

Sessions are page-local, not stored quiz history. Cancellation and timeouts preserve current local answers. Restart the backend after adding POST /api/resumes/quiz. The route bounds request size, concurrency, rate and provider duration. Edited PDF sources include bounded saved text replacements; handwriting and other unsaved or unsupported visual changes are not interpreted as applicant facts.

Sources: frontend/src/components/ResumeQuiz.jsx; backend/resumeQuiz.js; backend/aiAccess.js; backend/assistant.js.

Suggested events when campus events are enabled - Update 279

Calendar and Home now include Suggested events: up to six upcoming loaded TXST events within the next 30 days, ordered by date. Cards include the date, time, title, available location and a View event details action. Calendar opens the event's inspector; Home opens the public event page when a valid URL exists.

The Calendar suggestions section follows the TXST events source checkbox. Both surfaces omit events removed through hidden-event preferences and hide the section when all TXST events have been removed. Home follows the account-wide visibility setting rather than the Calendar page's temporary source checkbox. No invented placeholder events are used when the feed is empty. Loading and feed-warning states explain availability. Suggestions are chronological campus discovery, not AI recommendations or a claim of personalized suitability.

Sources: frontend/src/components/SuggestedEvents.jsx; frontend/src/components/HomePage.jsx; frontend/src/components/CalendarPage.jsx; frontend/src/hooks/useTxstEvents.js.

Keep a log of attended events - Update 280

Open Calendar and find Attended events below the suggestions section. The attendance log remains available even when campus suggestions are hidden. It is an account-owned record independent of whether a public event remains in the source feed.

- Select Log an event for an event outside the platform, or select a calendar event and choose Log attendance to prefill its title, location and source identity.
- Enter the actual date attended and optional notes or takeaways. The date control is limited to today or earlier; if a future event is selected, the form defaults to today and you must confirm the correct attendance date.
- Save attendance to persist the record to your account. Name, date, location and notes are stored; this does not duplicate the event into your calendar.
- Search the log by name, location, date or notes. Edit changes an existing record.
- Remove opens an inline confirmation. Removing a log entry leaves its calendar event unchanged. Calendar-linked records have duplicate-source protection.

The backend registers GET/POST /api/event-attendance and PUT/DELETE /api/event-attendance/:id, with validation and account-scoped queries. EventAttendance stores timestamps plus the attendance fields. Full account reset removes these records. Restart the backend to register the model and routes. The attendance log is not the project update logbook and does not automatically claim that you attended every past event.

Sources: frontend/src/components/EventAttendance.jsx; backend/eventAttendance.js; backend/accountReset.js; backend/server.js.

Calendar event description scrolling - Update 282

Click an event in the selected day's side list to automatically scroll up to its details and description. The same event can be clicked again to refocus it. Month and agenda event selection also use the shared selection helper, and suggested-event clicks use the same focus behavior.

Selection first updates the date and event, then schedules focus after the browser can render the selected inspector. Focus does not independently scroll the page; a single explicit scroll positions the details. Reduced-motion preferences use immediate movement instead of smooth scrolling. Scroll margins leave room for sticky navigation. Pending animation frames are canceled on unmount. This improves navigation without altering the event's content or attendance state.

Sources: frontend/src/components/CalendarPage.jsx; frontend/src/components/CalendarPage.css.

PDF replacement fonts and consistent export - Update 272

When editing existing PDF text, the editor prefers available embedded font programs. Font names with subset prefixes such as SFXSQO+CMMI12 are normalized for matching. Computer Modern variants, including Math Italic, can use bundled KaTeX-derived compatible font files when the source font cannot be loaded. Preview and PDF export use the same replacement bytes rather than previewing one font and silently exporting a generic standard font for those matched families.

If a missing font is reported, Use font file accepts a TTF or OTF file up to 10 MB. The supplied bytes are stored with matching text annotations and reloaded for later preview/export. The operation applies to matching font-name annotations already in the editor; it is not a system-wide font installation. Standard font fallbacks remain for unmatched fonts. Exact original outlines require the original usable font data, and replacement text still needs glyphs supported by the selected font. Compatible substitution cannot guarantee identical design or line breaks.

Sources: frontend/src/lib/pdfReplacementFonts.js; frontend/src/lib/pdfExistingText.js; frontend/src/lib/pdfTextStyle.js; frontend/src/components/PDFEditor.jsx.

A real website engine inside the study phone - Update 271

The ordinary browser version cannot override external X-Frame-Options or Content-Security-Policy frame restrictions. The optional desktop host offers Website inside phone, which displays external HTTPS pages as native Electron WebContentsView content positioned over the phone's page viewport. It avoids opening a separate website window for normal native phone navigation.

- Start the existing configured backend and frontend.
- In desktop, install dependencies once with npm install, then launch with npm start.
- Sign in, open the phone with Alt+P, choose its browser or shortcut and enter an HTTPS site.
- Use Website inside phone mode for native tabs. Back/forward follows actual website history; zoom changes browser zoom. Closing or suspending a tab releases its native view.
- Phone resizing, hiding, minimizing and switching apps update visibility and bounds. Saved sessions restore addresses and wait for Resume; they do not preserve unsaved website forms after a view is destroyed.

Remote pages have sandboxing, context isolation, web security and no Node.js or app preload bridge. Website cookies persist in separate account partitions on this device; account reset clears that account's website data. Device permissions and downloads are disabled. Website requests for new windows are redirected within the current native phone tab; flows that require special pop-ups or approved clients may fail. Explicit Open in browser actions still launch the system browser. Chess.com and third-party sign-in compatibility were not verified. This is an optional host, not a packaged installer or bundled backend. See desktop/README.md for configuration.

Sources: desktop/main.cjs; desktop/preload.cjs; frontend/src/components/NativePhonePage.jsx; frontend/src/components/PhoneBrowser.jsx; frontend/src/components/StudyPhone.jsx.

Login failure messages and recovery - Update 273

Login and registration now read API responses defensively rather than calling response.json unconditionally on empty output. The frontend checks that a successful response contains a token and user identity, handles bounded request timeouts, normalizes submitted email addresses, and avoids storing an incomplete session. Malformed saved user JSON is discarded so it does not block startup.

The Vite development proxy returns a JSON 503 message when the backend on port 5000 is unavailable. The backend checks JWT_SECRET and database readiness before authentication queries are queued. Restart both services after updating. If sign-in still fails, configure the existing environment requirements and restore database/API connectivity; improved parsing does not turn an unavailable backend into a successful login. No credentials are printed or stored by these error messages.

Sources: frontend/src/components/Auth.jsx; frontend/src/contexts/AuthContext.jsx; frontend/src/lib/apiResponse.js; frontend/vite.config.js; backend/server.js.

CHAPTER 03

Product overview

Purpose

AssignmentTracker is a full-stack student workspace connecting tasks, calendars, documents, study material and career preparation. It brings related work into one account while retaining clear distinctions between source records, linked files and the views that display them.

Main workflows

- Plan academic work with tasks, deadlines, folders, prerequisites, attached evidence and deletion recovery.
- Combine task deadlines, personal appointments, Canvas imports and public events in a calendar with categories, recurrence and supported undo.
- Store documents, create typed notes or blank notebooks, annotate PDFs, copy text and use local mathematical tools.
- Ask questions with selected workspace/file context, save study questions and prepare bounded edits to supported open documents.
- Save job postings and tailor resumes or letters using existing source material, reviewed replacements and optional employer research.
- Arrange tools in a customizable workspace, inspect task relationships visually and navigate through global search.

Core technology

The React/Vite frontend provides interactive views and document tools. An Express API handles authentication, validation, persistence, external retrieval and AI requests. MongoDB stores account-owned records, file data and transaction-backed recovery/history. Local utilities handle operations such as Markdown conversion, supported extraction, arithmetic and plotting; Gemini provides optional language generation or recognition.

Design priorities

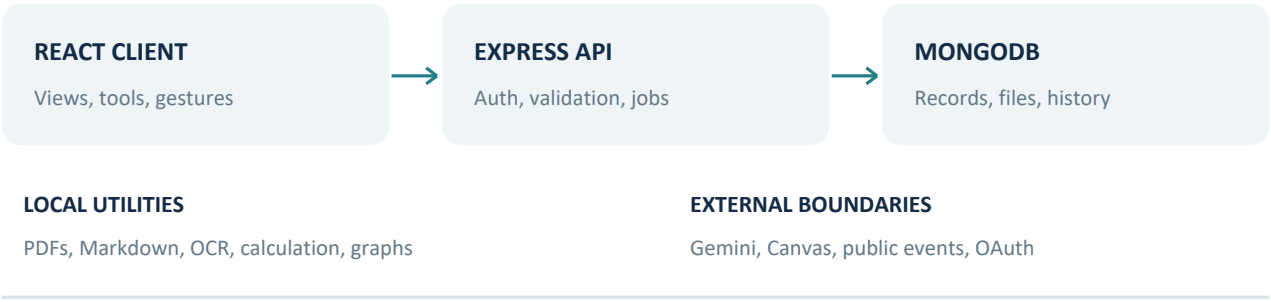
Related entities use stable references so a task, event or folder can link the same document. Destructive operations have workflow-specific recovery paths. AI capabilities are restricted by account permissions and supported editor actions. Public-source imports retain their provenance and expose freshness or collection failures.

Scope

The application is not a real-time collaborative editor, an unrestricted autonomous agent, a complete PDF word processor or a universally accurate job scraper. Its supported workflows and practical limits are documented in the following chapters.

CHAPTER 04

Architecture: the whole system



The main request path

Browser action -> React state -> authenticated /api request -> Express validation -> Mongoose owner-scoped query -> MongoDB -> response -> React refresh or optimistic update.

The Vite development server runs on port 3000 and proxies /api to the backend on port 5000. Hash routes such as #/tasks and #/documents let reloads and Back/Forward navigation work without a server route for every page. App.jsx coordinates shared tasks, folders, documents and the mounted editor. Feature-specific components and backend modules implement the individual workflows.

Where computation happens

- Browser: document viewing, annotation gestures, calculator/graph evaluation, Markdown rendering and PDF conversion, local job-file parsing/OCR, and interface state.
- Server: authentication, database writes, ownership checks, transaction history, remote website retrieval, AI requests, encryption of saved AI credentials and OneNote integration.
- Database: user accounts, tasks, folders, files, calendar records, chats, study items, jobs, generated drafts and recovery/index records.
- External services: Gemini, Canvas, public TXST event discovery, F1 schedule data and optional account-linking providers.

Three useful distinctions

UI state is not necessarily persistent data. A selected page or open menu is temporary; a task update is persisted. Browser preferences are not automatically account-wide settings. Public source caches are not private account records.

This is a modular application on one backend, not a fleet of microservices. Some coordination is per process, so multiple backend instances would require shared locking, rate limits and caches. MongoDB transactions require Atlas or a replica set.

Design rationale: Interactive rendering stays close to the browser, while credentials and persistence stay on the server. Structured IDs represent relationships, and transactional snapshots support recoverable mutations. Coordination becomes more complex as the feature set grows.

Sources: frontend/vite.config.js; frontend/src/App.jsx; backend/server.js; backend/calendarHistory.js.

CHAPTER 05

Data model and persistence

Core records

- User: identity, password hash, preferences, AI access, encrypted Gemini credential, calendar settings, workspace layout and connected-account metadata.
- Todo: title/text, status, description, deadline, folder, ordering, external source identifiers, links and prerequisite IDs.
- Folder and PDFFolder: separate task/document hierarchies, name, color, optional description and attachment IDs.
- PDF: the legacy collection name for all document formats; stores names, MIME type, binary data, annotations, tags, folder membership, task relationships, original backup and Trash state.
- CalendarEntry: personal appointments, date markers and sports events, linked task/document IDs, category, dates and optional recurrence grouping.
- AssistantConversation: messages, source links, file references, settings, title, archive/pin state and generation revisions.
- SavedJob and StudyItem: job evidence and structured starred learning material.

Supporting records

TaskDeletionRecovery preserves deleted task groups with expiration. CalendarChange records before/after changes. Generated resume and letter drafts are temporary records. DocumentSearchIndex stores extracted searchable content. OAuth state records are short-lived and single-use.

Why references matter

A document can be linked from several tasks or folder descriptions without uploading another binary copy. Unlinking does not delete the file. Stable task IDs make restored task references usable again. When a folder is deleted, its contents and child folders are promoted rather than destroyed.

Storage tradeoff

Keeping binary files in MongoDB makes ownership and backup behavior straightforward at this size, but large documents, backups and annotation arrays can approach document and memory limits. A production evolution would store binaries in object storage, retain metadata in MongoDB and stream uploads/downloads. That is a proposed improvement, not the current architecture.

Sources: backend/server.js; backend/folderAttachments.js; backend/taskDeletionRecovery.js; backend/assistantConversations.js; backend/globalSearch.js.

CHAPTER 06

Setup, configuration and deployment boundaries

Run the main application

From the repository root, install dependencies in backend and frontend with npm install. Copy backend/.env.example to backend/.env and configure MONGODB_URI and a strong JWT_SECRET. Start backend with npm run dev in one terminal and frontend with npm run dev in another. Open http://localhost:3000. Use Node.js 18 or newer as required by the existing backend code; choose a supported runtime for your own deployment.

Features with extra setup

- Transactions: use MongoDB Atlas or a replica set for undo, reordering, folder operations, attachments and other transaction-backed writes.
- AI: save a Gemini key in Account, or configure GEMINI_API_KEY on the backend. GEMINI_MODEL selects the provider model; compatible model availability and quotas must be verified in the provider account.
- Credential encryption: configure a stable AI_KEY_ENCRYPTION_KEY, or understand the fallback derived from JWT_SECRET. Changing these secrets can invalidate stored encrypted keys.
- OAuth: configure GitHub/LinkedIn client credentials, exact callback URLs and public application/API URLs. Production account linking expects HTTPS with a same-host /api proxy.
- OneNote: native preservation needs desktop OneNote on the Windows backend machine; compatibility conversion has its own Python requirements.

Prepare the workspace

Prepare an account, a small PDF, a typed note, sample tasks and a saved job. Verify optional integrations before depending on them. Local notes and PDF tools remain useful when network services are unavailable. Keep environment files, credentials and private documents protected.

Deployment instructions in the repository describe hosting options; they do not establish that a public deployment is currently running. No installation, backend start or live integration check was performed for this handbook.

Sources: backend/.env.example; frontend/vite.config.js; backend/package.json; frontend/package.json; SETUP.md.

CHAPTER 07

Home, navigation and global search

Home and navigation

Home is the signed-in landing page. It displays open-task/deadline counts, a focus list, completion progress, a seven-day agenda, recent documents and career shortcuts. Actions open the underlying task, file or calendar item. Empty and loading states reflect available records instead of invented activity.

The compact navigation keeps Home, Workspace, Tasks, Calendar and Documents prominent. Career groups Resumes, Cover letters and Saved jobs. Tools contains the assistant, Study stars, Visualization, Canvas and keyboard shortcuts. The profile menu exposes Account and sign out. Smaller layouts use expandable navigation; dropdowns support keyboard focus and dismissal.

Exhaustive search, with explicit coverage

Ctrl/Cmd+K opens global search, including from editors and fullscreen PDF mode. It searches page features, shortcuts, tasks, document names/tags/content, personal events, jobs, chat messages, study material, folder paths/descriptions and calendar categories. Public campus events are included through their existing loading hook. Trash is optional. Results route to the relevant editor, task, event, conversation or folder.

Document content indexing runs in bounded worker threads. The index version tracks saved file updates; outdated index text should not be treated as current. Reindex rebuilds the account's extracted text. Pagination prevents returning every result in one response. Search includes supported PDF text/annotations, DOCX, workbook and text content; it does not magically OCR every scanned image or index unsaved edits, audio speech or all embedded media.

Design rationale: Search results retain entity types and IDs so navigation can resolve the correct destination. Bounded workers isolate extraction from request handling. Saved content must be indexed before its newest text is searchable, and unsupported formats have explicit coverage limits.

Sources: AppNavigation.jsx; HomePage.jsx; GlobalSearch.jsx; backend/globalSearch.js; backend/searchTextWorker.js.

CHAPTER 08

Tasks: everyday planning

What the user can do

- Create tasks in Inbox or a chosen folder, edit titles and descriptions, and set deadlines and supporting links.
- Advance between To Do, In Progress and Completed; reopen completed work.
- View all tasks or a folder, search/filter the visible work, and choose sorting such as deadline, status or custom order.
- Expand cards or open task details for richer editing, connections and attachments.
- Use compact task titles by default and toggle full wrapping when needed.
- Select multiple tasks for bulk deletion, or delete an individual task with the trash control.
- Disable the separate In Progress column in Account -> Task display. Those tasks remain alongside To Do tasks and retain their status.

Dragging and ordering

Custom order supports drag and drop. A compact floating preview and insertion line communicate placement. Dropping across columns can change status. Pointer visuals are updated with requestAnimationFrame rather than causing a full React render for every pointer move. Wheel/trackpad scrolling remains available, and edge scrolling helps reach distant destinations. Saving errors roll back optimistic ordering; Escape cancels a drag.

Deletion recovery

Successful task deletions create account-owned recovery snapshots for seven days. Undo deletion appears below the page heading, works after reload and can be repeated for earlier groups. It covers individual deletions, bulk selections and clear actions. Restoration preserves IDs, details, dates, order and dependency links; a missing original folder sends the task to Inbox. Account reset intentionally clears these snapshots.

Design rationale: A status column is a view rather than a task identity. Hiding it must not delete tasks or make them inaccessible. Transaction-backed deletion snapshots are persistent recovery records, unlike a temporary client-side notification.

Sources: frontend/src/App.jsx; useTaskPointerDrag.js; TaskDeletionUndo.jsx; TaskDisplaySettings.jsx; backend/taskDeletionRecovery.js.

CHAPTER 09

Task relationships and attached evidence

Prerequisites and follow-ups

Task connections let one task come before or after another, including across folders. Links save immediately and are visible from either endpoint. They are advisory: the app does not block starting or completing a task just because prerequisites remain open. Duplicate/self-links are rejected or deduplicated, and circular chains are flagged for review.

The visualization turns prerequisites into arrows. Sequence layout arranges acyclic chains; cycles and unresolved sequences are handled separately. Deleting a task removes its dependency references, and recovery can restore appropriate connections.

Files and media

Task details and expanded editors have Files & media. Upload multiple files, drop files onto the panel, choose a document destination folder, or attach an existing library document. Images, audio and video support authenticated previews. Other formats open through the document editor or can be downloaded. Each upload is limited to 10 MB.

An existing document can be referenced by several tasks. Insert in description adds a stable document reference rather than copying the binary or freezing its filename. Library rows and the task map expose related tasks/documents. Unlinking an attachment keeps the library file, and Trash makes a reference unavailable until the document is restored.

Relationship design

Stable IDs let renamed documents resolve correctly. Updating a relationship differs from uploading a file, and removing a link does not destroy shared data. Prerequisites are advisory rather than enforced scheduling constraints, preserving flexibility while exposing cycles.

This is not automatic project scheduling or critical-path optimization. Do not claim that the graph guarantees a feasible plan, detects every business dependency, or assigns resources.

Sources: TaskDependencies.jsx; TaskAttachments.jsx; frontend/src/lib/taskDependencies.js; backend/taskDependencies.js; backend/taskDeletionRecovery.js.

CHAPTER 10

Folders, descriptions and folder media

Organizing work

Tasks and documents have separate folder trees. Folders support names, colors and up to 12 levels of nesting. Breadcrumbs and full paths make location visible. Create a child with New subfolder or choose a parent in the creation form. Moving a folder moves its descendants logically; ownership and cycle/depth checks run on the server.

Deleting a folder keeps its directly assigned items and children by promoting them to its parent or the top level/Inbox/Unfiled. Folder deletion is not recursive file destruction.

Descriptions

Each folder and subfolder has an optional 2,000-character description. Add it during creation, or select the folder and choose Add description / Edit description. Emptying and saving removes the description. The description appears below its path and is searchable.

Images and files

Files & media beneath the description supports multiple uploads, drag and drop, clipboard images while focused, and linking existing Documents. A folder supports 50 attachments with a 10 MB per-file upload limit. Previews for images, audio and video load on demand; files can be opened, downloaded through the shared save flow or unlinked. Missing/trashed files remain removable unavailable links.

Uploaded files and their attachment relationship are created in one transaction. Document-folder uploads are stored in that folder; task-folder uploads are stored in Unfiled. Unlinking keeps the file in Documents. Folder deletion also preserves library files.

Uploading into a new folder

The document upload panel includes Save in folder -> New folder. Enter a name, description and parent, then Create & select folder. The destination changes without clearing the chosen file, name or tags. Finish or cancel folder creation before uploading. A created folder remains if the upload is later cancelled or fails.

Sources: backend/folderHierarchy.js; backend/folderAttachments.js; FolderHierarchyControls.jsx; FolderDescription.jsx; FolderAttachments.jsx; UploadFolderPicker.jsx.

CHAPTER 11

Calendar: a unified view of several sources

What appears

Month and agenda views combine task deadlines, imported Canvas assignments/events, personal appointments, date markers, sports entries and public TXST events. The calendar is a projection of several record types, not one copied list. Opening a task deadline returns to its original task. Undated tasks are shown separately until assigned a date.

Search and source/completion filters help narrow the view. Today, month navigation and agenda selection make it usable beyond the current day. Personal appointments support start/end times or all-day ranges, locations, notes and completion; markers represent dates or inclusive date ranges.

Connected records

Personal entries can link tasks and existing documents or upload supporting media. Save commits the links with the event. Cancelling a form discards unsaved links but can leave uploaded files in Documents. Deleting/unlinking an event preserves its linked tasks and documents.

Dates are not all the same

Timed events represent instants displayed in the chosen account time zone. All-day dates represent calendar dates and must not shift to the previous/next day because of a UTC conversion. This distinction also matters for multi-day ranges and imported ICS end dates.

Remove versus delete

Remove this event targets exactly one event ID. Excluding a task deadline does not delete its task or modify Canvas. Excluding a public event does not alter the source. Deleting a personal calendar record is a different operation. Matching-name removal is a separate reviewed action; similar titles alone do not trigger bulk removal.

Design rationale: Source identity and calendar visibility are separate. A calendar-only removal can leave the underlying task or external source untouched.

Sources: CalendarPage.jsx; CalendarAttachments.jsx; backend/calendarEntries.js; backend/calendarLinks.js; frontend/src/lib/calendar.js.

CHAPTER 12

Calendar: recurrence, colors, categories and undo

Personal organization

Users can create named categories, choose category colors and filter by category. Individual color overrides remain available. Categories are account-owned, validated against events and cannot be deleted while still in use. A category is organization, not an external event source.

Recurring meetings

Repeat controls can generate grouped meetings at minute, hour, day, week or month intervals, including chosen weekdays. The implementation stores a finite set of entries with a shared series ID and rule metadata. Creation accepts 2-500 meetings; it is not an infinite scheduler. Series operations can update shared details or remove the group; per-event actions still target an individual item unless the group action is selected.

The creation batch is transactional. Linked task/document ownership and category validity are checked. Time-zone and calendar arithmetic are important around daylight-saving transitions and month boundaries; review the preview before saving.

Similar-name removal

Remove similar opens a reviewed selection across loaded events. Exact matching normalizes case, accents and punctuation. Similar matching uses shared words/token overlap rather than an AI classifier. Uncheck matches to keep. It operates on loaded IDs, not an eternal rule that removes every future event with the same words.

Undo boundaries

Undo and Ctrl/Cmd+Z outside text fields reverse the latest supported saved calendar change: creation, edits, deletion, series/import batches, visibility and related settings changes. History persists in MongoDB and rejects conflicting newer changes rather than overwriting them. Calendar undo consumes an entry and has no redo. Task edits outside calendar history, source changes at TXST, unsaved text and account reset are not universal undoable operations.

Sources: CalendarRecurrence.jsx; CalendarCategories.jsx; CalendarUndo.jsx; backend/calendarSeries.js; backend/calendarHistory.js; backend/calendarCategories.js.

CHAPTER 13

Public TXST events and the scraper

User experience

Opening Calendar starts loading upcoming/ongoing public TXST events alongside personal work. Refresh TXST requests another collection; progress and errors remain visible. Filters and an event inspector expose names, dates, descriptions, locations, organizations, categories, themes, benefits and source/RSVP links when available.

Public events are not inserted as personal tasks or appointments. An account can hide individual IDs, remove similar loaded events, hide all current/future campus events, or restore exclusions. Those preferences survive refreshes without changing the public source.

Collection design

The backend scraper requests public CampusLabs discovery search pages using skip/take pagination, deduplicates IDs, then retrieves details sequentially. It uses delays, timeouts and transient-error retries. JSON preserves normalized data plus raw listing/detail fields; CSV offers a flatter export. A partial collection contains errors and a complete flag instead of pretending every result was retrieved.

Calendar requests share an in-process public cache. An open calendar can trigger refresh after an hour; manual refresh has a cooldown. Partial results can retain prior data, so stale events are possible until a complete refresh. A restart clears this cache, and multiple backend processes do not automatically share it.

Coverage boundary

"All events" means all publicly returned events within the selected search, not private, restricted, unpublished or deleted university records. Website endpoints can change. The app does not bypass login restrictions, RSVP on behalf of the user, or guarantee live upstream compatibility.

CLI examples: npm run scrape:txst in backend; optional --from, --to, --out and --csv arguments. Pass an explicit time-zone offset when a UTC midnight date boundary is not intended.

Sources: backend/scrapeTxstEvents.js; backend/txstCalendar.js; backend/TXST_EVENTS.md; frontend/src/hooks/useTxstEvents.js.

CHAPTER 14

Sports and F1 season import

What the feature does

Calendar supports manual sports entries with sport, league, team/opponent and normal appointment details. F1 has a dedicated schedule importer, rather than a promise to scrape every sport automatically.

Choose a season, preview the returned schedule, and select session types. Controls distinguish free practices, qualifying, sprint qualifying, sprints and Grands Prix. Only buttons make it easy to select one group. Individual rows can also be included or excluded before importing. Choose an optional calendar category for the resulting records.

How it avoids duplicates

The backend normalizes schedule sessions into stable external IDs. Re-import skips already imported sessions rather than creating duplicate appointments. The importer can categorize selected existing entries without overwriting the user's edited times/details. A batch import and its supported changes participate in calendar undo.

Data quality

The F1 schedule comes from Jolpica's public data. Sessions with known start times use those timestamps; finish times may be estimates. Missing times can appear as all-day placeholders. Public data and race schedules can change. Imports are snapshots, not a subscribed live calendar that continuously rewrites the user's events.

Design rationale

Schedule import depends on normalization and idempotency: identifying the same session on a later import, separating session types, handling missing timing data and preserving user edits. Uncertainty is visible rather than replaced with an invented exact end time.

Example workflow: preview a season, choose only Grands Prix, assign a Sports category and import. A subsequent import should identify already imported sessions. Check the live source and review timing information before relying on the schedule.

Sources: F1SeasonImport.jsx; backend/f1Calendar.js; CalendarPage.jsx; backend/calendarHistory.js.

CHAPTER 15

Canvas synchronization and ICS import

Canvas connection

The Canvas section accepts connection information and supports manual synchronization of assignments and calendar events. Synced data feeds task lists and the unified calendar. Source IDs/types distinguish assignments from events. Updates refresh source titles/dates while preserving local task state, notes, folder choices and dependency relationships.

Pagination is part of correctness. The backend follows same-origin pagination links rather than assuming one response contains all results. Calendar contexts are batched instead of sent in an unbounded request. Partial failures are reported so one successful course does not hide another failed course.

ICS files

Users can import an exported calendar file. The parser handles folded lines, UID-based updates, UTC and named-zone timestamps, and all-day end dates. Invalid dates are skipped, not replaced by today. Floating times without a zone use the implementation's UTC fallback. Existing ICS recurrence rules are not expanded into every occurrence locally.

Limits and privacy

These are imports/syncs into the application, not two-way editing of Canvas. A status change in the app should not be described as completing an assignment in Canvas. Credentials stay out of AI workspace context, but Canvas credentials and their storage still deserve a separate security review; do not assume they have the same encryption path as saved Gemini keys.

Design rationale: Imports keep source identity separate from local state so updates do not erase user work. Partial retrieval is reported as partial rather than empty. Scheduled queues, monitored retries and broader conflict policies are potential extensions.

Sources: frontend/src/App.jsx; backend/canvasPagination.js; backend/canvasTasks.js; backend/calendarICS.js; backend/server.js.

CHAPTER 16

Document library and supported formats

Library features

Upload files up to 10 MB, rename the display name, add tags, organize folders/subfolders, search and sort. Select individual or multiple records to move or send to Trash. Drag documents onto folder destinations. Restore from Trash or use the library's deletion undo. Linked tasks are visible from file rows.

The legacy PDF collection stores multiple file kinds. Opening a document routes by extension/MIME type instead of forcing every file into a PDF renderer. Content saves preserve an uploaded original backup; original plus edited content must fit within the 15 MB storage rule. Large or unsupported content remains downloadable.

Format behavior

- PDF: rendered pages, annotation tools, page operations, selected source-text replacement and export with annotations.
- Images: converted to an editing surface; annotated output can be exported as an edited image.
- TXT/code/Markdown: UTF-8 text editing; notes-tagged text and Markdown use the writing studio.
- DOCX: editable text with original package-based serialization, retaining supported document formatting and embedded package content. The text editor is not a full Word layout preview.
- CSV/TSV: quoted-field-aware cell grid; larger cell sets can fall back to text editing.
- XLSX: select worksheets, edit values/formulas, add visible rows/columns. Excel recalculates formulas on open; this is not an in-browser Excel formula engine.
- Browser-supported audio/video: playback, download and replacement.
- Other formats: storage, download and same-format replacement; no arbitrary native editor.

Design rationale: Format dispatch gives each file a defined capability set. Preserving an original DOCX package differs from rebuilding a generic document; original backups provide a recovery path when edits are limited.

Sources: DocumentLibrary.jsx; DocumentEditor.jsx; frontend/src/lib/documentFormats.js; frontend/src/lib/wordTemplateEditor.js.

CHAPTER 17

Typed notes and the writing studio

Creation and structure

Documents -> New notes offers Blank, Lecture, Cornell study, Meeting, Project brief and Daily journal templates. New notes use editable Markdown. Existing notes-tagged text and Markdown also open in the studio. Templates provide headings, prompts, tables and checklists; they do not claim to automatically know the user's course or meeting.

Writing features

- Write, Split and Read views; live formatted Markdown with mathematical notation.
- Headings, bold, italic, strikethrough, inline code, bullet/numbered lists, checklists, quotations/callouts, links, tables, dividers and display equations.
- Heading outline with navigation, word/character counts, approximate reading time and checklist completion counts.
- Literal find/replace-all with optional case matching; bounded undo/redo history.
- Enter continues a list; Shift+Enter inserts a plain newline. Tab inserts two spaces; Shift+Tab can leave the field.
- Modern, Book and Mono display fonts and four text sizes; preferred view/font/size are remembered locally.
- Focus mode removes surrounding clutter. Save stays available; Escape dismisses Find or exits Focus.

Saving and exports

Manual Save persists content; optional Autosave saves after a writing pause. Session drafts support recovery when available, and unsaved content triggers leave-page protection. Failed autosave leaves the note unsaved and displays an error; Save can retry.

Download Markdown preserves editable source. Export PDF opens the existing converter with current text, including unsaved edits, and creates a separate downloadable PDF. Writing font/size are display preferences, not a promise that PDF export inherits those settings. The export preview has its own typography.

Design rationale: Markdown provides durable, portable source text and local previews without arbitrary editable HTML. Toolbar actions insert syntax rather than implementing a complete word processor.

Sources: NoteStudio.jsx; noteTemplates.js; DocumentEditor.jsx; MarkdownPDFCreator.jsx.

CHAPTER 18

Document creation: blank, Markdown and AI

Blank handwritten notebooks

Create a notebook with 1-500 empty pages in US Letter or A4. It opens directly in the PDF editor for handwritten notes. This differs from typed Markdown notes: it is a page-based annotation workflow.

Markdown to PDF

Paste Markdown or import .md/.markdown/.txt, choose a name and A4/Letter, then inspect the preview. Headings, lists, code, tables, quotations and KaTeX mathematics render locally. Export preservation handles typed spacing and wrapped lines, with special preparation for math so fraction bars and glyphs are not naively clipped.

Long content is split into bounded conversion sections and merged into one PDF. This avoids treating a browser canvas-size limit as a user instruction to manually split every document. The result has page numbering and can be downloaded or saved into the library and opened for annotation. It is a rendered export, not a guarantee of fully selectable/vector typeset text. External images remain links. Conversion and file-size limits still apply.

Create with AI

Describe the desired document. Gemini returns a draft title/text for review. After editing the draft, create a PDF that opens in the editor. Generated text is placed as editable annotations on base pages, so use the PDF editor's Download to include it in the finished file. Plain-text generation does not constitute full LaTeX typesetting or image generation.

The generation flow uses the configured AI key and generation permissions. The basic document creator sends the supplied description, not the entire document library automatically. Review facts and unsupported characters before relying on the result.

Design rationale: AI drafts content while local code creates and paginates the artifact. Markdown conversion is a separate deterministic workflow with no model-token requirement.

Sources: DocumentLibrary.jsx; AIDocumentCreator.jsx; MarkdownPDFCreator.jsx; markdownPdfExport.js; backend/generatedDocuments.js.

CHAPTER 19

PDF editor: annotation fundamentals

Tool modes

Select text is a reading/copying mode. Pen draws freehand ink. Highlight adds highlighter strokes. Text inserts or edits supported text objects. Move selects and repositions inserted objects. Erase affects annotations, not original page content.

Annotation controls include named/custom colors, numeric and slider sizes, and pen pressure where supported. Pen widths span 1-40 pt and text controls 6-144 pt. Toolbars group settings rather than showing every control continuously. Undo/redo apply to annotation changes; Save persists them and Download burns visible edits into an exported PDF or edited image.

Eraser choices

Whole stroke removes matching strokes. Precision removes intersecting portions of vector strokes. Objects removes whole text/image annotation objects. Shape, size and pen/highlighter filters let the user choose the target. A footprint indicates the active area. Erasing is undoable and does not remove printed PDF content.

Straight-line assistance

A long, sufficiently straight pen stroke can snap to a line at release. Keep / Undo lets the user retain or revert it; immediate Ctrl+Z first restores the original freehand points. The detector rejects short strokes, bends and backtracking and does not change highlighter strokes. It can be disabled.

Images, snippets and inserts

Insert includes files/images, PDF pages, shapes, image tables and stamps. Tables accept cell entry or pasted spreadsheet cells up to 12 rows by 8 columns, then insert as an image. Stamps have editable text and presets such as APPROVED, DRAFT, REVIEWED and CONFIDENTIAL. Shapes and other inserts can be moved and resized. PDF merge accepts multiple files, import ranges and before/after/start/end placement. Copy region captures a selected page region as a snippet; paste it elsewhere and move/resize it. This transfers rendered content, not necessarily the original editable source objects. The selection/move tools and keyboard nudges help place inserts precisely.

Design rationale: The annotation model is separate from the base document, supporting editing and undo without rewriting the PDF for every pointer sample. Export composes the annotations into a portable artifact.

Sources: PDFEditor.jsx; PDFInsertPanel.jsx; PDFEraserControls.jsx; pdfEraser.js; pdfStraightLine.js; insertGeometry.js.

CHAPTER 20

PDF text: editing, fonts and copying

Editing existing selectable text

Choose Text and click a supported source text run. The editor extracts text and style and creates a replacement annotation. Detection considers font family, size, bold/italic, color, opacity, spacing, baseline and horizontal scaling. Available embedded font programs can be reused for preview/export; the catalog and same-page peer information help matching when direct metadata is limited.

Leaving text unchanged preserves the original rendering. Unsupported characters and missing/custom subset fonts can require an approximation or stop export rather than silently substitute every glyph. Independently skewed, vertical or image-only text is not equivalent to ordinary horizontal selectable text.

A crucial security distinction

Source text is visually covered and replaced. It is not necessarily removed from the PDF content stream. This is editing by overlay, not secure redaction. A white cover also cannot reconstruct arbitrary photographs or colored backgrounds behind the old text. Use a dedicated redaction workflow when underlying source content must be removed.

Copying text

Select text mode exposes native selection without drawing. The text layer includes different fonts, sizes and positions plus relevant saved text annotations, and suppresses covered source runs for copying. Copy all text extracts across pages, including pages not currently rendered. Join wrapped lines groups soft line wraps into coherent paragraphs while preserving useful structure such as lists, headings, contact lines and code-like content.

Practical limitations

PDFs store positioned glyphs and drawing commands, not necessarily semantic paragraphs. Character geometry and reading order can be ambiguous, particularly in columns or complex layouts. Scanned pages need OCR; selection improvements do not create text that does not exist. Always review copied output from difficult documents.

Sources: pdfExistingText.js; pdfTextStyle.js; pdfFontCatalog.js; pdfPeerFont.js; PDFSelectableLayer.jsx; pdfCopyText.js.

CHAPTER 21

PDF pages, views and productivity tools

Page organization

Navigate with the page field, thumbnails or previous/next arrows, including fullscreen arrows. Insert a blank page after the current page, import page ranges from another PDF, or duplicate supported pages through Insert -> Pages. Drag thumbnails to reorder. Delete a page when more than one remains. Page mutations remap annotation page numbers to preserve their association with content.

Page operations are different from an ink gesture. They save document structure and may clear the annotation undo stack; do not promise every deleted/reordered page can be restored with the normal ink Undo button. A clean copy creates a separate document without editor annotations. Changes already burned into the underlying PDF are not removable merely by clearing the annotation array.

Reading and comparison

Single-page and continuous views support different reading habits. Continuous view activates a page using viewport visibility and waits around active gestures/text edits to reduce flicker. A side page can show a reference page while another page is edited. Fit width, zoom, view rotation and page navigation remain available.

Interface customization

Fullscreen reduces surrounding UI while keeping annotation tools, navigation and access to the assistant. Shrink/Expand changes toolbar density. Toolbar position can be Automatic, Top, Bottom, Left or Right; side positions are vertical. Dark pages invert the displayed page/annotations and thumbnails; saved/exported colors remain original. Photos also invert in this view.

Small utilities

Clock visibility, stopwatch and countdown timer support focused work. They are interface utilities, not time-tracking records or server-scheduled reminders. Check their visible controls for start/pause/reset behavior; a closed or suspended browser is not a guaranteed alarm service.

Sources: PDFEditorChrome.jsx; PDFEditor.jsx; backend/pdfPages.js; PDFEditorClock.jsx; PDFStopwatch.jsx; PDFCountdown.jsx.

CHAPTER 22

Pen, touch, mouse and rendering performance

Input arbitration

Scroll and Draw are explicit finger modes. One-finger behavior follows that mode; two fingers can pan and pinch. Pen contact takes priority over touch. A second contact can cancel a tentative finger stroke before starting navigation. The implementation uses browser pointer types and gesture ownership, not a machine-learning palm classifier.

Native stylus button events can map the main side button, secondary button and eraser tip to supported tools/actions. The settings page provides graphical controls and remembered preferences. Browser/hardware support varies: a web app cannot receive a hardware button event that the OS or browser never exposes.

Pinch zoom anchors around the finger center, uses a composited preview during the gesture and commits a final render scale. Panning can coast using recent velocity with elapsed-time friction, bounded speed/duration and scroll-edge limits. New input cancels momentum; reduced-motion preferences disable added glide. Native wheel/trackpad scrolling should not receive a second inertia effect. Mouse interactions still support drawing, selection and the applicable drag/navigation controls.

Avoiding per-sample React work

Completed ink and live ink use canvas layers instead of one SVG/React element per pressure segment. Live strokes paint incrementally; completed strokes append where possible. Eraser hit tests skip distant strokes using cached bounds. Continuous pages render lazily, canvas backing resolution is bounded, and a worker can serialize annotation JSON. Autosave waits for active gestures to pause and skips stale queued snapshots.

Attribution and honesty

Pointer-input helpers were adapted from Excalidraw v0.18.0 under MIT; the repository includes source notes and a license. This is not embedding the entire Excalidraw editor or implementing hardware palm detection. Say the architecture aims to reduce work and bound resources; do not claim zero lag or a measured speedup without profiling.

Sources: usePDFTouchInput.js; PDFInkLayer.jsx; annotationSerializer.js; PDFStylusSettings.jsx; frontend/src/lib/vendor/excalidraw/SOURCE.md.

CHAPTER 23

Mathematics, calculator, function scanning and graphs

Local calculation

The PDF editor has a calculator and region-based tools. Drag a box around a printed arithmetic expression to extract its text and evaluate it locally. Supported syntax includes common arithmetic, parentheses, powers, percentages, roots and constants. Recognition normalizes common PDF symbols into the parser's supported syntax.

Scan function -> calculate lets the user select a function, review its recognized expression, insert an input value and compute the result. This is not general symbolic algebra or proof generation.

Recognition modes

Text only uses local text extraction. Auto attempts local reading and can fall back to Gemini when needed. Handwriting / AI uses the selected image crop. The crop includes relevant visible annotations rather than sending an unrelated whole workspace screenshot. AI access and recognition permission must allow it.

AI is used to transcribe into supported math syntax; deterministic local code evaluates the result. A visibly incorrect transcription can still produce a mathematically correct answer to the wrong expression, so review is part of the workflow.

Graphing

Graph selected region or type an explicit function of x. Adjust X/Y limits, inspect the plot, insert it as a movable image annotation or download a PNG. Supported functions include common trigonometric, logarithmic, exponential and rounding functions. Trigonometry uses radians, log is base 10 and ln is natural logarithm.

Plots use finite sampling. They can miss narrow features or behave poorly around discontinuities; implicit relations and arbitrary multivariable equations must be rewritten. This is not a complete computer algebra system.

Design rationale: Recognition transcribes expressions and a restricted local parser evaluates them. This supports handwriting assistance without running arbitrary JavaScript or delegating every arithmetic result to a language model.

Sources: PDFCalculator.jsx; PDFGraph.jsx; calculator.js; pdfCalculation.js; pdfGraph.js; mathSelectionImage.js.

CHAPTER 24

OneNote imports and preserving handwriting

Entry points

Upload a .one section or .onepkg notebook through the normal document upload, or open Import OneNote. Review source items, select the pages/sections to keep, inspect the preview and choose Save & open notes. The destination folder and tags are retained. The result is a PDF for the platform editor.

Native path

The preferred path asks desktop OneNote for Windows on the backend machine to publish source pages as PDFs. This preserves OneNote's native rendering of handwriting and annotated printouts. Imports serialize access to the shared application and retry temporary COM busy conditions. If page discovery fails, a whole-section native export can preserve the section's PDF sheets.

The native preview can report source ink counts when available and offers Download original OneNote PDF for comparison. Saving a single native result preserves its exact bytes; combining selections retains all PDF sheets.

Compatibility path

An explicit compatibility option uses the separately installed Python converter. It is cross-platform in intent but can lose unsupported ink, objects or layout. SVG ink is dimensioned/rasterized for conversion. The app should not silently switch from failed native export to a lossy result.

Limits and troubleshooting

Uploads are 10 MB, source pages are bounded, and resulting PDFs must fit the library limit. Desktop OneNote must open normally under the backend Windows account, complete sign-in/setup and unlock protected sections. Keep original files; flattened imported ink cannot be erased as individual platform strokes. New annotations are editable, but source OneNote objects are not.

Design rationale: Native OneNote export uses the original application renderer to preserve visual content more faithfully than a compatibility parser. The resulting dependency on Windows/desktop OneNote is explicit; direct PDF upload is an alternative.

Sources: OneNoteImport.jsx; backend/oneNoteImport.js; backend/scripts/export_onenote.ps1; backend/scripts/import_onenote.py; backend/requirements-onenote.txt.

CHAPTER 25

AI assistant: conversations and user experience

Access and controls

Open the dedicated assistant page or the floating panel; Alt+J toggles/focuses it and Alt+8 opens its page. The panel can be moved, resized, filled to available space or reset. Page and panel dimensions have separate local preferences. Fullscreen PDF editing can expose the assistant beside the document.

Conversation management

Saved account-owned chats support automatic titles, rename, pin, archive/restore, search, pagination, private direct links, Markdown export and permanent deletion. Copy messages or code blocks. Edit an earlier question in a branch or continue from an earlier answer without overwriting the original conversation. Regenerate replaces an answer only after success; failed regeneration preserves it. Continue helps with truncated replies, and Jump to latest helps when reading history.

Messages render Markdown, tables, code and KaTeX inline/display math, with sources and context summaries. Prompt starters and Concise/Balanced/Detailed response settings help guide use. Attachment choices and settings persist per conversation. Star a saved question to capture it and its available answer in Study stars.

History is bounded

The server supplies saved history, not an arbitrary browser-authored transcript. Conversations have a message cap, while only a bounded recent slice is sent to the model. Older messages remain viewable; a long chat does not mean the provider sees all of it on every request. Generation leases and revision checks reduce overlapping replies from multiple sessions.

Capability boundary

The assistant answers using selected context and can prepare a narrow set of supported edits for the currently open editor when authorized. It is not an unrestricted autonomous agent: it cannot arbitrarily mutate tasks or browse the entire internet as a general tool.

Sources: AssistantChat.jsx; AssistantMarkdown.jsx; useAssistantResize.js; backend/assistantConversations.js; backend/assistant.js.

CHAPTER 26

AI context, credentials, access and diagnostics

Explicit context

Per-chat switches choose structured workspace information, visible page text and the open file. Users can select explicit file attachments. Supported PDFs/media may be sent as provider inline inputs; text/DOCX/workbooks use extraction with bounds. Visible-screen context is text, not an automatic screenshot of the whole computer. Unsupported content gets a notice instead of an invented extraction.

Account-level controls

AI access controls can disable AI globally or restrict profile, tasks/folders, calendar, library metadata, resumes/letters, notes, other document bodies, screen text, history, live editor reading, editor writes, generation and recognition. Backend enforcement overrides per-chat switches. Restricting categories can suppress screen/history context to avoid leaking protected information through a different channel. It cannot recall data already sent, remove historical provider copies or prevent a user from deliberately typing private information into a prompt.

Saved credentials

Personal Gemini keys are encrypted server-side with AES-256-GCM and account-bound associated data; a stable server secret derives the encryption key. The browser receives masked status, not the stored raw key. A configured server fallback can apply when a user has no personal key. Saving a key does not prove quota, billing or model availability.

Failure handling

The backend bounds request concurrency, context and rate. Timeouts abort requests; it does not indiscriminately retry provider calls. Diagnostics distinguish authentication, permission, model, quota, throttling, safety/context, network and service failures. Trace IDs, stage, timings, safe provider details and retry countdowns make errors actionable. A deliberate connection test consumes provider usage; a configuration/status refresh need not call Gemini.

Design rationale: AI controls rely on capability scoping, data minimization and validated output. Prompt instructions alone cannot guarantee trustworthy model behavior. Provider policies and availability remain external dependencies.

Sources: backend/aiAccess.js; backend/assistantCredentials.js; backend/assistantDiagnostics.js; AccountAIAccess.jsx; AssistantDiagnostics.jsx.

CHAPTER 27

AI document editing and PDF cleanup

Editing the open document

Enable Open file context and allow edits, then request a specific change. The browser's editor bridge exposes only the mounted editor's supported live text IDs and revision. Supported actions include replacing supplied text, adding a PDF text annotation on the current page and navigating pages. Plain-text notes can accept an entire-document text replacement. Closed attachments, Word files and spreadsheets are not arbitrary writable targets through this bridge.

The provider prepares an editor-actions JSON block. Backend validation checks action type, count, IDs, text length, coordinates and page. The browser rejects stale results when the document/page/version has changed. Reopening a saved chat does not replay actions. A PDF batch becomes one annotation undo step and uses annotation persistence; text notes still require saving or their enabled autosave.

Clean up PDF

The cleanup tool accepts user comments, extracts supported text from the current page or bounded document scope, and asks for proposed corrections. The user reviews individual proposals and can select/edit them before applying. Supported corrections become text edits in one undo batch; it does not magically repair arbitrary handwriting, diagrams or document semantics.

Why both layers validate

The server validates proposed model actions and the browser checks whether the target is still current. Together these checks handle malformed output and the race where editing continues while a request is in flight. The model receives no direct database or filesystem access.

Restrictions remain real: PDF replacements are visual overlays, font coverage can fail, scans may lack text, and context is bounded. Do not describe this as full autonomous document editing, secure redaction or guaranteed correctness.

Sources: frontend/src/lib/editorBridge.js; backend/assistantEditor.js; PDFCleanup.jsx; backend/pdfCleanup.js; DocumentEditor.jsx.

CHAPTER 28

Jobs: links, screenshots, PDFs and local extraction

Saved job records

The Jobs page stores a posting's title, company, location, type, salary, dates, URL, description, requirements, responsibilities and benefits. Users review fields before saving and can return to the original posting. Saved roles provide context to cover-letter and resume generation.

Import by link

The backend retrieves public posting pages and uses structured job data and site-specific/general parsing where available. Outbound retrieval validates public URLs, addresses, redirects, timeouts and body size. Handshake receives special handling for publicly available posting data and a pasted-content fallback when the page depends on login or client-side content.

Universal does not mean unrestricted: the server cannot access a private authenticated session merely because the user can see that page in another tab. The app should ask for pasted text or a captured document rather than bypass access restrictions.

Import from files

Upload an image or PDF of the posting. Local mode uses PDF text/layout extraction or browser OCR for raster content, then heuristics to classify header fields and sections. The layout-aware parser helps separate title, company and location from a long description. Manual correction remains available. AI mode is an explicit alternative and can consume tokens; the local extractor is deterministic/OCR-based, not a privately hosted language model.

Implementation rationale

Structured extraction is attempted where available, source evidence is retained and fields remain reviewable. Recovering semantic fields from arbitrary layout differs from extracting raw text. Labeled evaluation sets and per-field precision/recall would support a broader quality assessment.

No claim of perfect imports is warranted. A screenshot can omit context, OCR can confuse symbols and columns, and different job boards can change their markup.

Sources: JobsPage.jsx; JobFileImport.jsx; localJobFile.js; frontend/src/lib/jobParsing/; backend/jobs.js; backend/jobFileImport.js; backend/jobExtraction.js.

CHAPTER 29

Cover letters: reference voice and template fidelity

Workflow

Upload and store one or multiple cover-letter files in the career section. Select a template/reference letter and a saved role or supplied job details. The generator uses source material to propose role-specific replacements, with existing wording and factual evidence prioritized. Review the generated draft in a continuous text box rather than disconnected sentence chips.

Previous letters guide voice and phrases as well as layout. The workflow reports evidence, warnings and verified phrase reuse so the user can judge whether the result sounds like them. Existing documents are evidence, not permission to invent new achievements or copy every irrelevant sentence into the output.

Preserve the original artifact

The generation model supplies structured text replacements, not the final PDF/DOCX binary. Local/template logic places reviewed replacements into supported original regions. PDF edits reuse compatible font/layout slots where possible. DOCX edits preserve the original ZIP package and XML structure rather than rebuilding everything with a generic font. If a replacement does not fit or the font lacks a glyph, the app should expose the limitation instead of silently shrinking or substituting everything.

Save and error review

Save a separate copy and open the editor, leaving the reference intact. Review/acknowledgement gates and descriptive warnings help avoid treating an AI draft as a final application. Expandable errors include stage, category, safe provider/application details, HTTP status, trace ID and suggested next step. Compact prompts and bounded references reduce unnecessary context; provider latency can still vary.

Design rationale: The model proposes language changes while deterministic code retains format ownership. Existing source phrasing and supported facts take priority over generic generated wording.

Sources: CoverLettersPage.jsx; CoverLetterGenerator.jsx; CoverLetterError.jsx; backend/coverLetterPrompt.js; backend/coverLetterGeneration.js; coverLetterPdfLayout.js; wordTemplateEditor.js.

CHAPTER 30

Resumes and company-specific tailoring

Resume storage and generation

The resume section stores PDF/DOCX resumes that are also accessible in Documents. Choose a source template, relevant previous resumes and a role. Resume-specific instructions prioritize source-supported skills, projects and experience while preserving identity, employment/education chronology and official titles. It is not a license to invent qualifications for a job description.

The resume generator shares document-generation infrastructure with letters but uses a different prompt and purpose. References for resumes come from resume material. Review replacements and warnings before saving a separate copy.

Annotated templates

The workflow accounts for saved editor text annotations as well as source PDF text. Covered original runs should not be duplicated into the prompt as if still visible. Saved annotation-backed regions can be updated while retaining other ink/images/highlights. A template version check prevents a draft from being applied against a newer source file. Handwriting/images retained as visual content do not become applicant facts unless actually interpreted by a supported path.

Company website research

Both resume and letter generation can use the employer's official public website. Confirm/enter its URL; known job-board/ATS URLs are not substitutes for the official company site. Bounded same-host public pages provide business/product context, with source links and excerpts in review. Employer context helps prioritize relevant existing projects, not fabricate experience or affiliation.

Public pages are untrusted input. Retrieval is bounded, validated and separated from applicant documents; the app does not send the user's resume to the employer website. Website failure falls back with a warning to role and source documents. Cleanup-only letter mode does not need research.

Design rationale: Employer facts and applicant facts have distinct roles. Research may change relevance but must not change what the applicant actually did.

Sources: backend/resumePrompt.js; backend/companyResearch.js; backend/coverLetterGeneration.js; tailoringTemplateRuns.js; ResumesPage.jsx.

CHAPTER 31

Custom workspace and task visualization

A workspace of tools

Workspace provides saved workflow tabs containing draggable/resizable windows. A window can hold tool tabs for Tasks, Agenda, Documents, Task map, Notes, Focus timer and the assistant entry point. Users can group/split tools, rename tabs, minimize/maximize windows, tile, choose templates, adjust density/grid/snapping, lock layout and use fullscreen mode.

Planning, Research and Minimal templates are starting arrangements, not copies of the user's underlying data. Scratchpad notes are workspace content, separate from standalone document notes. Export/import JSON transfers supported layout and workflow-note state; it does not export every task/document binary. A focus timer is local runtime behavior, not a durable scheduled job.

Layout and notes save to the account with revisions. A browser draft helps recover failed/offline saves. Conflict handling offers reload or save-this-version choices rather than silently overwriting a newer layout. Layout undo/redo is distinct from calendar or document undo.

Visualization

The task map groups by folder/status or uses planned sequence. It shows actual task dependencies and document relationships. Search, status/folder/overdue filters, pan/zoom, fit/reset, minimap and an inspector support exploration. Users can open tasks/documents, edit and advance status. Completion, workload and deadline charts are derived from the same records.

Dragged map positions are temporary view adjustments, not persistent changes to task order. Filters can hide graph edges to excluded nodes without deleting the relationships.

Design rationale: Workspace tools and the task graph project the same underlying records. They do not require a second task database for display.

Sources: WorkflowWorkspace.jsx; WorkspaceWindow.jsx; WorkflowTools.jsx; useWorkspaceLayout.js; backend/workspaceSettings.js; TaskVisualization.jsx; taskVisualization.js.

CHAPTER 32

Study stars and learning workflows

Saving material

Tools -> Study stars is a dedicated collection for questions and topics. A saved user question in the AI assistant has a star control. Starring captures the question, its available following answer and a source conversation reference. It is not automatically a synchronized mirror of all future revisions to that answer.

Users can also create manual questions/topics with a subject, answer/study material and personal notes. Edit or unstar items, search their contents, filter by Questions/Topics/To review/Studied and mark a review status. Reveal answer/material supports active recall instead of displaying everything immediately. A source conversation link returns to the saved chat when still available.

Ownership and persistence

Study records are account-owned and stored in MongoDB. When starring a saved conversation question, the backend loads the owning conversation and message rather than trusting an arbitrary client-supplied answer. A source key prevents repeated stars for the same source question from creating duplicates. Study material participates in global search and account reset.

An example learning flow

Create lecture notes, attach a source PDF, ask the assistant to explain a concept, star the difficult question, then revisit it in Study stars and reveal the answer only after attempting recall. Manual topic entries can organize material that did not originate in the assistant.

Design rationale: Study stars turns saved explanations into reusable review material. The current implementation tracks collection/review status; it is not an adaptive learning or spaced-repetition algorithm.

Sources: StudyPage.jsx; backend/study.js; AssistantChat.jsx; frontend/src/lib/study.js.

CHAPTER 33

Account, authentication and security boundaries

Account features

Account includes profile/activity information, time-zone selection, action-confirmation preferences, task-column visibility, AI credentials and access controls, connected accounts and clean account reset. Time-zone settings can follow the device, select an IANA zone, search cities or request an IP-based suggestion. These optional lookups depend on external services; VPNs can affect IP location.

Authentication

Registration/login use bcrypt password hashing and signed JWTs. The implementation issues seven-day tokens and the frontend persists authentication state in localStorage. Protected routes verify the token and query by user ownership. File IDs alone are not authorization.

The existing CORS and token-storage choices should not be presented as a completed production security review. localStorage tokens are exposed to successful script injection. Potential improvements include a reviewed cookie/token strategy, tighter origins, stronger upload/content policies, account abuse controls and dependency maintenance.

Connected identities

GitHub/LinkedIn connections link a provider profile to the current account; they do not replace application login or automatically import repositories, contacts or social activity. Authorization uses expiring single-use state tied to a browser cookie; GitHub also uses PKCE. Provider tokens fetch profile information and are discarded. Disconnect invalidates pending attempts and removes the local link; provider-side revocation is separate.

Destructive account reset

Reset requires the current password and RESET confirmation, uses a transaction, and clears application data including recovery/history while preserving login identity. It is not undoable. Per-process write guards must be coordinated before deploying multiple backend instances. Do not reset an account containing valuable data without retaining any required exports first.

Sources: AuthContext.jsx; backend/server.js; backend/accountConnections.js; backend/accountReset.js; TimeZoneSettings.jsx; ConfirmationSettings.jsx.

CHAPTER 34

Downloads, recovery and the meaning of Save

Choose a download destination

All platform export paths share a save helper: library files/originals, PDF/image output, notes, letters/resumes, graphs, OneNote native previews, chats and workspace JSON. When supported in a secure context with user activation, the native Save As picker lets the user select folder and filename. After long preparation, a ready dialog provides a fresh Choose location click.

Cancelling does not trigger a second download automatically. Write failure keeps the prepared file available for another attempt. Browser download is an explicit alternative. Browsers without this API require their own Ask where to save each file setting; the site cannot force an arbitrary local path. Fullscreen editors use a top-layer save dialog.

Recovery is deliberately scoped

- Tasks: seven-day server snapshots, including bulk groups, with stable-ID restoration.
- Calendar: durable before/after history for supported calendar mutations, conflict-aware undo, no redo.
- Documents: Trash and restoration; content originals where supported; text/workbook session drafts.
- PDF annotations: undo/redo and annotation save, distinct from structural page operations.
- Notes: bounded edit history, drafts and optional autosave.
- Workspace: layout history, account revisions and local draft/conflict handling.

None of these promise recovery of data from before the feature existed, externally changed sources, expired snapshots, account reset, every deleted page or files already exported to another program.

Design rationale

Save changes the app's persistent record; Download creates a local artifact. Some formats consist of a base file plus separately editable annotations. Export is explicit, and each workflow has its own recovery model rather than a universal rollback mechanism.

Sources: frontend/src/lib/download.js; DocumentEditor.jsx; PDFEditor.jsx; TaskDeletionUndo.jsx; backend/calendarHistory.js; useWorkspaceLayout.js.

CHAPTER 35

Separate daily-events site and QR poster

What it is

The daily-events directory is a standalone frontend project, separate from the authenticated AssignmentTracker app. It summarizes campus events for residents who scan a QR code. It uses plain HTML/CSS/JavaScript and a published events.json file; no backend or build step is required to serve that directory.

Features

- Month calendar and selected-day agenda with event details/source links.
- Previous/next month, Today, date selection and Next event navigation.
- Search, categories and a free-food filter based on source fields.
- America/Chicago date boundaries, overlap handling and midnight rollover when viewing Today.
- Dark mode by default with a remembered light-mode toggle.
- Automatic JSON reload approximately every minute, retaining the last loaded collection if reload fails.
- Manual file import for local preview; automatic reload resumes through Reload events.
- Public URL entry, locally generated QR code, printable light poster and SVG QR download.

Data freshness is not automatic scraping

The browser's minute refresh only reads events.json. The separate scraper must update/publish that file. Importing a file in one visitor's browser does not update the hosted dataset. The interface shows update time and warnings for stale/partial exports. Inspect the actual hosted file before claiming it contains current events; README passages about its included data reflect different stages of the project.

Running and explaining it

From the repository root: python -m http.server 8080 --directory daily-events. Open http://localhost:8080. Use a real public URL in a printed QR code, not localhost. Host the full directory including vendor assets and events.json.

Design rationale: The standalone site separates public read-only event distribution from the private student workspace. Static hosting simplifies delivery, but fresh-data publishing remains a separate responsibility.

Sources: daily-events/README.md; daily-events/index.html; daily-events/app.js; backend/scrapeTxstEvents.js.

CHAPTER 36

Guided end-to-end workflow

1. Start with an overview

Open Home to review task counts, upcoming dates and recent documents. Select an item to open its underlying record. Use a disposable workspace or non-sensitive sample material while learning the controls.

2. Organize a task

Create a task, assign a folder and deadline, and add a prerequisite. Drag it to another position or status. Delete a disposable task and restore it with Undo deletion. The recovery snapshot preserves task identity and relevant relationships. Account -> Task display controls whether In Progress has its own column.

3. Connect it to the calendar

Open the task's deadline from Calendar. Create a personal appointment and link a document. Check the time zone, then distinguish removing a single displayed event from deleting a personal entry or selecting matching-name removals. Use calendar undo for supported saved changes.

4. Create notes and supporting material

Create a lecture-note template, add a heading, list and equation, and switch between Write, Split and Read. Use the outline to navigate. Add a description and a small image to its folder; the attachment remains a document-library record.

5. Work with a PDF

Open a small PDF, draw and undo a stroke, select/copy text, and open a side page. Try a local calculation or graph. Source-text replacement uses visual overlays; dark-page inversion is a display setting. Test pen and touch behavior on the actual device before relying on it for substantial work.

6. Use AI and career tools selectively

Review Account -> AI access before sharing content. Open a saved conversation or request a supported edit with appropriate context enabled. In Career, save a job and review a tailored document against source evidence. Keep provider diagnostics available when requests fail, and review all generated facts and layout warnings.

7. Return to related work

Use global search to reopen the task, file, event or conversation. Explore Workspace, Visualization and Study stars as alternate views or collections. The separate daily-events QR site is a public distribution workflow with its own data-publishing step.

CHAPTER 37

Technical questions: architecture and data

Why React, Express and MongoDB?

React supports the stateful editor and composed panels. Express provides a straightforward API layer, and Mongoose supplies a shared record model for owner-scoped data. MongoDB accommodates evolving task/document metadata. Relationships and transactions still require explicit coordination; binary storage is a candidate for future separation.

Which parts extend beyond CRUD?

Record management supports more complex workflows: document geometry, gesture ownership, safe recovery, cross-entity relationships, extraction/indexing, AI context boundaries and template-preserving exports. Each has its own validation, state transitions and failure handling.

How is account data isolated?

The token establishes identity and protected database queries include the owning user. Relationship writes validate ownership of referenced documents, tasks and folders. An object ID alone does not grant access. Negative authorization cases should be covered by the validation plan.

Why use transactions?

A deleted task and its recovery record must commit together. The same requirement applies to a calendar mutation and its history, or a folder attachment upload and its link. Transactions avoid partial relationship/history writes and require a transaction-capable MongoDB deployment.

What happens with concurrent edits?

Some workflows use revisions or per-account transaction locks, including chats, layouts and calendar history. Other paths need additional concurrency review. The application does not provide general-purpose real-time collaboration or identical conflict handling for every record type.

How could the system scale?

Possible extensions include object storage for binaries, a job queue for slow conversion/indexing, broader pagination, shared public caches/rate limits and operational observability. Memory, response time and queue delay should guide those changes. These are proposed improvements rather than deployed infrastructure.

CHAPTER 38

Technical questions: PDF, input and performance

Why are PDFs difficult to edit?

A PDF often describes positioned glyphs and graphics rather than semantic paragraphs. Replacing text can require font coverage, line layout and background reconstruction. Supported source-text replacement uses overlays; it does not securely redact the underlying content.

How are annotations preserved across pages?

Annotations retain page references and document-space coordinates. Page mutations remap those references. Continuous view separates passive pages from the active overlay and waits for matching geometry. Late callbacks must not apply a different page's dimensions to the active page.

How is repeated drawing work reduced?

The implementation uses live/completed canvas layers instead of a large React/SVG tree for every pressure segment. It appends where possible, indexes stroke bounds for erasing, caps backing resolution and defers serialization/save work. Quantitative improvements require frame-time and memory measurements.

Is palm rejection AI-based?

No. Input handling relies on pointer type, pen ownership and explicit touch modes. Two-pointer gestures support navigation. Browser/hardware events limit classification. Small input helpers were adapted from Excalidraw under MIT with attribution; this is not hardware-level palm recognition.

What is the tradeoff of a custom annotation model?

A narrower model provides direct control over annotation behavior and exports, but text layout and device compatibility require substantial maintenance. A mature full editor library could replace some of that work if its capabilities and integration costs fit the product.

What should be profiled?

Input-to-paint latency, browser long tasks, stroke-count scaling, memory at high zoom/continuous view, export duration, extraction time and worker overhead. Use defined device/document/stroke workloads for comparisons rather than interpreting a visual impression as a benchmark.

CHAPTER 39

Technical questions: AI, extraction and trust

Which tasks benefit from AI?

Language tailoring, explanations and difficult handwriting recognition can benefit from model assistance. Ordinary search, supported job extraction, arithmetic after parsing, and Markdown-to-PDF conversion use local or deterministic paths. Separating those paths limits latency, token use and unnecessary sharing.

How are unsupported claims reduced?

Generation uses source evidence, bounded tasks, structured replacements and user review. Employer context does not authorize invented applicant facts. For mathematics, recognition and evaluation are separate. These controls reduce risk but do not guarantee error-free output.

Can source content contain malicious instructions?

Documents and websites are untrusted input. Prompt instructions state that boundary, while validated capabilities provide stronger restrictions: known target IDs, bounded actions and browser revision checks, without arbitrary execution. Prompt-injection behavior still needs adversarial evaluation.

Why can job extraction fail?

HTML, OCR and PDF layout provide incomplete signals. A page may require login or dynamic rendering, and a screenshot may omit important fields. Multiple input paths and editable fields support recovery. Missing information should remain an explicit gap rather than be fabricated.

How can latency be improved?

Use structured extraction first, trim irrelevant references, bound employer research, choose an appropriate configured model and avoid blind retries. Provider latency should be measured separately from parsing, retrieval and export. Prompt changes cannot resolve quota or network failures by themselves.

How should quality be evaluated?

Use consented/redacted fixtures to measure per-field job accuracy, evidence/phrase compliance, layout overflow, math transcription and action rejection. Compare modes on equivalent inputs and record model versions and conditions. Existing source and test files are not evidence of recorded pass results.

CHAPTER 40

Engineering case studies

Recoverable task deletion

Problem: deleting a task group can remove details and dependency relationships. Approach: store task/dependency snapshots in a separate expiring collection and delete the active records in the same transaction. Restoration preserves stable IDs and validates surviving folders. Boundary: recovery expires and account reset clears it.

Heavy annotation workloads

Problem: accumulated strokes increase rendering, hit-testing and serialization work. Approach: use canvas layers, incremental painting, cached bounds, worker serialization and bounded backing resolution. Tradeoff: high-zoom display resolution is capped to control resources. Performance conclusions require measured workloads.

Writing and template fidelity

Problem: generated career documents can diverge from source voice, evidence or layout. Approach: reuse supported source phrases, generate structured replacement regions, preserve template-owned formatting and report font/overflow failures. User review remains necessary; the application does not guarantee application outcomes.

OneNote annotation fidelity

Problem: a compatibility parser can omit ink or rearrange annotated printouts. Approach: prefer native OneNote PDF export, preserve sheets, expose diagnostics and make the compatibility path explicit. Tradeoff: native fidelity introduces a Windows/desktop dependency and cannot recover content missing from the uploaded source.

Shared lessons

- Preserve identity and provenance when data appears in multiple workflows.
- Commit related state changes together when partial success would be misleading.
- Distinguish interactive previews from portable exported artifacts.
- Isolate slow or uncertain work and show actionable failure states.
- Attribute adapted components and review their licenses.
- Record verification conditions before asserting performance or accuracy improvements.

CHAPTER 41

Limitations, testing plan and roadmap

Current boundaries worth stating

Browser capabilities vary for save pickers, stylus events, codecs and touch input. Public endpoints, company pages and AI quotas can fail or change. PDF text replacement is not redaction; fonts, scans and complex layouts are limited. Job extraction and model output require review. OneNote fidelity depends on native export and source completeness. Recurrence is finite, and undo is scoped. Some caches/locks are process-local. Large binary documents and annotation arrays pressure browser/database resources.

A meaningful validation plan

- Unit fixtures: date-only versus timed conversions, daylight-saving recurrence, parser normalization, dependency handling, page remapping and font/text logic.
- API integration: ownership failures, rollback on transaction errors, expired recovery, duplicate import, stale revisions and file limits.
- Browser/device checks: keyboard and focus behavior, large PDF scrolling, mixed pen/touch gestures, fullscreen dialogs, copy text and exported output.
- AI/extraction evaluation: known source evidence, malformed actions, prompt-injection attempts, OCR errors, per-field accuracy and formatting overflow.
- Operational checks: memory, response time, provider timeouts, logs without secrets, multi-instance coordination and restore/recovery procedures.

These are recommended checks, not checks executed for this handbook. The repository contains targeted test files, but their presence does not establish pass status.

Prioritized evolution

First: stabilize critical workflows with representative fixtures and observability. Next: split large coordinator components, centralize data fetching and formalize API schemas/conflict rules. Then: move binaries to object storage and slow jobs to a queue, with shared coordination. Finally: broaden extraction support or add review scheduling only after measuring demand and quality.

Engineering priority: Stabilize and measure the existing workflows before expanding feature breadth. Reliability, observability and maintainability are the next milestones.

CHAPTER 42

Troubleshooting and operational recovery

App does not load

A Vite/Babel syntax error points to a source file and line; inspect that location rather than hiding the overlay. Confirm frontend dependencies and server startup. The frontend's /api proxy must reach the backend. An empty/non-JSON response often means a wrong route, stopped backend or stale running version.

Saved tasks or folders fail

Check account/session and the server's response. Transaction errors often indicate a standalone MongoDB deployment instead of a replica set/Atlas. A folder/file ID must belong to the signed-in account. Restart the backend after schema/route updates; refresh the frontend after UI updates.

Calendar is empty

Check source filters, hidden-event preferences, dates/time zone and upstream loading/error state. A minute browser refresh of the separate daily-events site is not a new scrape. Do not invent placeholder events to disguise missing source data.

AI or tailored generation fails

Expand diagnostics and identify the stage: credentials, provider request, template extraction, research, layout or saving. Verify key/model/quota yourself. Retry only when appropriate; shorten excessive input, use a clear template and check font/overflow warnings. A timeout is not automatically proof of a bad prompt.

File output looks wrong

Compare original, editor view and exported file. Confirm annotations were saved/exported, check the page count and font warnings, and distinguish native OneNote output from compatibility conversion. Dark-page display colors should not be mistaken for export colors.

Working without external services

Use saved records and local notes/PDF tools when external integrations fail. Inspect the failure stage and diagnostic before retrying a provider. Keep a copy of the source PDF and final reviewed artifact. Account reset is a permanent data operation, not a routine troubleshooting shortcut.

CHAPTER 43

Source map and feature inventory

Frontend entry and shared infrastructure

App.jsx coordinates routes, tasks, documents and editors. AppNavigation.jsx and GlobalSearch.jsx handle navigation/search. AuthContext.jsx handles session state. documentFormats.js dispatches file kinds, download.js handles saving, editorBridge.js exposes mounted editor capabilities, and shortcuts.js defines the main shortcut registry.

Academic workflows

TaskDependencies.jsx, TaskAttachments.jsx, TaskDeletionUndo.jsx, TaskDisplaySettings.jsx and useTaskPointerDrag.js cover task relationships, evidence, recovery, layout and dragging. CalendarPage.jsx, CalendarRecurrence.jsx, CalendarCategories.jsx, CalendarAttachments.jsx, CalendarUndo.jsx and F1SeasonImport.jsx cover scheduling. FolderHierarchyControls.jsx, FolderDescription.jsx, FolderAttachments.jsx and UploadFolderPicker.jsx cover nested organization and media.

Documents and study

DocumentLibrary.jsx, DocumentEditor.jsx, NoteStudio.jsx, AIDocumentCreator.jsx, MarkdownPDFCreator.jsx and OneNoteImport.jsx are the major document entry points. PDFEditor.jsx and PDFEditorChrome.jsx coordinate PDF behavior; focused helpers cover ink, erasing, text, copy, math, fonts and gestures. StudyPage.jsx and backend/study.js implement starred material.

AI, career and workspace

AssistantChat.jsx plus backend/assistant*.js implement chat, context, diagnostics, credentials and permitted edits. CoverLetterGenerator.jsx and backend/coverLetterGeneration.js share tailoring infrastructure; resumePrompt.js, coverLetterPrompt.js and companyResearch.js separate purpose/evidence/research. JobsPage.jsx, job import modules and jobParsing helpers implement extraction. PrepPage.jsx collects career preparation tools; ResumeTemplateGallery.jsx and resumeTemplates.js create Word layouts; ResumeQuiz.jsx and backend/resumeQuiz.js provide interview practice. EventAttendance.jsx and backend/eventAttendance.js persist attended events. SuggestedEvents.jsx provides feed-based suggestions. NativePhonePage.jsx and desktop/main.cjs implement optional native phone browsing. WorkflowWorkspace.jsx and TaskVisualization.jsx provide additional projections of the same records.

Backend source of truth

server.js defines models and registers routes. Feature modules own transactional behavior, parsing and external calls. Consult the endpoint and shortcut appendices that follow for a source-derived inventory. Filenames in this guide are repository-relative identifiers, not claims that the guide exercised every path.

The separate daily-events project has its own README and script assets. Vendor source notes identify adapted Excalidraw input helpers. Review licenses and attribution for adapted components.

APPENDIX A

Keyboard shortcut reference

These entries come from frontend/src/lib/shortcuts.js. Scope and focus determine whether a shortcut is active; text-entry exceptions and open dialogs can suppress it. Ctrl means Ctrl on Windows/Linux or Cmd on macOS where supported. Labels are registry descriptions; account confirmation preferences still apply.

Companion

Keys	Action
Alt + P	Open or close the study phone beside PDFs and notes

Calendar

Keys	Action
Ctrl / Cmd + Z	Undo the latest saved calendar change outside text fields
N	Add a personal appointment
Shift + N	Add a date marker
T	Jump to today
[	Previous calendar month
]	Next calendar month
/	Search the calendar
Esc	Dismiss calendar form or details

Platform

Keys	Action
Alt + 9	Open the customizable workspace
Alt + 8	Open the AI assistant page
Alt + 7	Go to Calendar
Alt + 1	Go to Tasks
Alt + 2	Go to Documents
Alt + 3	Go to Canvas integration
Alt + 4	Go to Account
Alt + 5	Open the keyboard shortcuts page
Alt + 6	Go to task visualization
Ctrl / Cmd + K	Search the entire platform, including inside editors
Ctrl / Cmd + Shift + K	Open the command section
?	Open keyboard shortcuts
/	Focus the current page’s search or task input
Esc	Dismiss an open inline form or command section

Workspace

Keys	Action
Alt + N	Open the window and tool picker
Alt + T	Tile visible windows
Alt + F	Toggle full-screen workspace
Ctrl / Cmd + Z	Undo a layout change outside text fields
Ctrl / Cmd + Shift + Z	Redo a layout change outside text fields
Esc	Dismiss the tool picker/settings or exit full-screen workspace
Alt + Arrow keys / Alt + Shift + Arrow keys	Move a focused window header (Shift increases step)

Assistant

Keys	Action
Arrow keys / Shift + Arrow keys	Resize using a focused edge or corner handle (10 px steps; Shift for 40 px)
Home	Reset assistant size with a resize handle focused
Ctrl / Cmd + Shift + O	Start a new chat
Ctrl / Cmd + Shift + B	Toggle chat history
Ctrl / Cmd + Shift + F	Search saved conversations
Alt + J	Toggle the floating assistant; focus the composer on its page (works in editors)
Esc	Close the assistant panel

Visualization

Keys	Action
/	Focus task-map search
0	Fit the task map
=	Zoom into the task map
-	Zoom out of the task map
R	Reset the map layout
Esc	Clear task-map filters

Documents

Keys	Action
U	Upload a document
Shift + N	Create a folder
Ctrl / Cmd + A	Select all visible documents
M	Move selected documents
Delete	Move selected documents to Trash
R	Restore selected documents from Trash
F2	Rename the single selected document
Shift + R	Refresh the library
D	Download the single selected document

Tasks

Keys	Action
E	Edit the focused task
Space	Advance the focused task’s status
Delete	Delete the focused task (requires inline confirmation)
Ctrl / Cmd + S	Save active task or folder edits
N	Focus the new task input
Shift + N	Create a task folder

Canvas

Keys	Action
O	Choose an ICS calendar file
Ctrl / Cmd + S	Import the selected calendar file

Document editor

Keys	Action
Ctrl / Cmd + S	Save edits
Ctrl / Cmd + Shift + S	Download the edited document
Esc	Close the editor (unsaved edits are protected)

Visual editor

Keys	Action
Ctrl / Cmd + S	Save annotations
Ctrl / Cmd + Shift + S	Download with annotations
Ctrl / Cmd + Z	Undo annotation change
Ctrl / Cmd + Shift + Z	Redo annotation change
Ctrl / Cmd + Y	Redo annotation change (alternative)
V	Select the Move tool
T	Select the Text tool
P	Select the Pen tool
H	Select the Highlighter tool
E	Select the Eraser tool
I	Open or close the insert shelf
Page Up	Go to the previous page
Page Down	Go to the next page
=	Zoom in
-	Zoom out
0	Fit the document to the viewer
R	Rotate the view
Delete	Delete the selected inserted object (undoable)
Shift + =	Enlarge the selected insert
Shift + -	Shrink the selected insert
←	Move the selected insert left
→	Move the selected insert right
↑	Move the selected insert up
↓	Move the selected insert down
Esc	Close the editor and save pending annotations

Notes studio additionally handles Ctrl/Cmd+B, Ctrl/Cmd+I, Ctrl/Cmd+F, Ctrl/Cmd+Z, Ctrl/Cmd+Shift+Z and Ctrl/Cmd+Y in the writing field. Its Tab behavior differs from ordinary page navigation. See the notes chapter.

APPENDIX B

API implementation inventory

The following literal and template route declarations were extracted from backend JavaScript source. This is a navigation aid, not a tested OpenAPI contract. Parameterized registration templates retain ${...} notation; their concrete bases are explained below. Auth, body shape and capabilities must be checked in the named module. No endpoints were called.

accountConnections.js

Method	Path / registration template
GET	/api/account/connections
POST	/api/account/connections/:provider/start
GET	/api/account/connections/:provider/callback
DELETE	/api/account/connections/:provider

aiAccess.js

Method	Path / registration template
GET	/api/account/ai-access
PUT	/api/account/ai-access

assistantConversations.js

Method	Path / registration template
GET	/api/assistant/conversations
POST	/api/assistant/conversations
GET	/api/assistant/conversations/:id
POST	/api/assistant/conversations/:id/fork
PATCH	/api/assistant/conversations/:id
DELETE	/api/assistant/conversations/:id

assistantCredentials.js

Method	Path / registration template
GET	/api/account/ai-settings
PUT	/api/account/ai-settings
DELETE	/api/account/ai-settings

calendarCategories.js

Method	Path / registration template
GET	/api/calendar/categories
POST	/api/calendar/categories
PUT	/api/calendar/categories/:id
DELETE	/api/calendar/categories/:id

calendarColors.js

Method	Path / registration template
GET	/api/calendar/colors
PUT	/api/calendar/colors

calendarHistory.js

Method	Path / registration template
GET	/api/calendar/history
POST	/api/calendar/undo

calendarSeries.js

Method	Path / registration template
POST	/api/calendar-series
PUT	/api/calendar-series/:id
DELETE	/api/calendar-series/:id

coverLetterGeneration.js

Method	Path / registration template
POST	${base}/generate
POST	${base}/save-draft

eventAttendance.js

Method	Path / registration template
GET	/api/event-attendance
POST	/api/event-attendance
PUT	/api/event-attendance/:id
DELETE	/api/event-attendance/:id

f1Calendar.js

Method	Path / registration template
GET	/api/calendar/sports/f1
POST	/api/calendar/sports/f1/import

folderAttachments.js

Method	Path / registration template
PUT	${base}/:id/attachments
POST	${base}/:id/attachments

folderHierarchy.js

Method	Path / registration template
PUT	${base}/:id
DELETE	${base}/:id

generatedDocuments.js

Method	Path / registration template
POST	/api/documents/generate
POST	/api/documents/create-generated

globalSearch.js

Method	Path / registration template
POST	/api/search/reindex
GET	/api/search/tasks/:id
GET	/api/search

jobFileImport.js

Method	Path / registration template
POST	/api/jobs/extract-file

jobs.js

Method	Path / registration template
POST	/api/jobs/extract
GET	/api/jobs
GET	/api/jobs/:id
POST	/api/jobs
DELETE	/api/jobs/:id

mathRecognition.js

Method	Path / registration template
POST	/api/calculator/recognize

oneNoteImport.js

Method	Path / registration template
POST	/api/documents/import-onenote

pdfCleanup.js

Method	Path / registration template
POST	/api/pdfs/:id/cleanup

phoneReference.js

Method	Path / registration template
POST	/api/phone/compatibility
POST	/api/phone/reference

resumeQuiz.js

Method	Path / registration template
POST	/api/resumes/quiz

server.js

Method	Path / registration template
POST	/api/auth/register
POST	/api/auth/login
GET	/api/assistant/status
POST	/api/assistant/test
POST	/api/assistant/chat
GET	/api/account/preferences
PUT	/api/account/preferences
POST	/api/user/reset
GET	/api/calendar-entries
POST	/api/calendar-entries
PUT	/api/calendar-entries/:id
DELETE	/api/calendar-entries/:id
GET	/api/todos
POST	/api/todos
POST	/api/task-dependencies
POST	/api/todos/reorder
PUT	/api/todos/:id
POST	/api/todos/bulk-delete
DELETE	/api/todos/:id([a-fA-F0-9]{24})
DELETE	/api/todos/folder/:folderId
DELETE	/api/todos/clear-inbox
GET	/api/folders
PUT	/api/user/canvas
GET	/api/user/canvas
POST	/api/canvas/test
POST	/api/canvas/sync
POST	/api/documents/actions
GET	/api/pdfs
GET	/api/pdfs/:id
POST	/api/pdfs/:id/clean-copy
POST	/api/pdfs

Method	Path / registration template
PUT	/api/pdfs/:id/task-link
PUT	/api/pdfs/:id/content
GET	/api/pdfs/:id/original
PUT	/api/pdfs/:id/annotations
POST	/api/pdfs/:id/pages
DELETE	/api/pdfs/:id/pages/:pageNumber
PUT	/api/pdfs/:id
GET	/api/pdfs/:id/download
DELETE	/api/pdfs/:id
GET	/api/pdf-folders
GET	/api/health
POST	/api/calendar/import

study.js

Method	Path / registration template
GET	/api/study
POST	/api/study
PATCH	/api/study/:id
DELETE	/api/study/:id

taskDeletionRecovery.js

Method	Path / registration template
GET	/api/todos/deletion-history
POST	/api/todos/deletion-history/:id/undo

txstCalendar.js

Method	Path / registration template
GET	/api/calendar/txst-events
POST	/api/calendar/txst-events/refresh
PUT	/api/calendar/visibility

workspaceSettings.js

Method	Path / registration template
GET	/api/account/workspace-layout
PUT	/api/account/workspace-layout

Registration notes

Folder hierarchy and folder attachment templates register under /api/folders and /api/pdf-folders. Tailoring registers separate cover-letter and resume generation/save-draft paths. Route templates and helper-generated routes should be read with their registration calls in server.js; the literal inventory may omit routes constructed by helpers.

APPENDIX C

Technical quick reference

Area	Implementation	Boundary / tradeoff
The product	Connected academic and career workspace	Feature breadth does not prove reliability
Architecture	React -> Express -> MongoDB	Not microservices or real-time collaboration
Ownership	Token plus owner-scoped queries	IDs alone are not authorization
Undo	Workflow-specific snapshots/history	No universal rollback; reset is permanent
PDF editing	Base file plus annotations and overlays	Not secure redaction or full Word layout
Performance	Canvas, bounded work, workers	No measured speedup without profiling
Palm rejection	Pointer type and gesture arbitration	No hardware/ML palm classifier
AI use	Language/recognition with restricted context	Review required; provider dependency
Math	AI transcribes; local parser computes	Wrong transcription can yield wrong result
Career tailoring	Source evidence plus template fidelity	No invented facts or ATS guarantee
Job imports	Structured data, layout/OCR, manual review	No universal accuracy or login bypass
Events	Public source fetch, cache and identity	Upstream changes and freshness limits
Scaling	Storage/queues/shared coordination next	Proposed, not deployed infrastructure
Testing	Representative fixtures and metrics	Source inspection is not test execution

Operational checklist

Keep source documents, reviewed exports and recovery options available. Check integration configuration, protect credentials, review errors before retrying and distinguish implemented behavior from proposed improvements. Track adapted components and their license requirements.